NAME: RUDRA · S · DHOLE

ROLL NO: 25101A0022

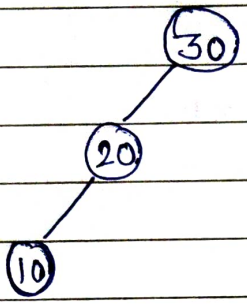
INFT- A

ASSIGNMENT - 07

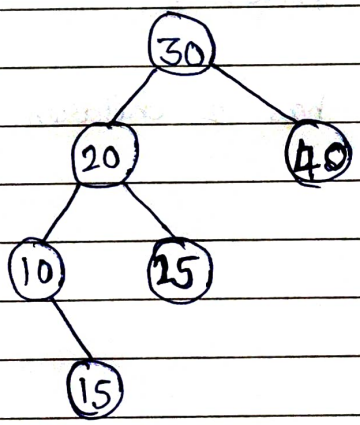
Q.1] The preorder traversal sequence of a Binary Search Tree (BST) is given as: 30, 20, 10, 15, 25, 40, 35, 50. What is the postorder traversal sequence of this tree? Derive your answer by drawing step-wise diagrams. (LO4.2).

Ans

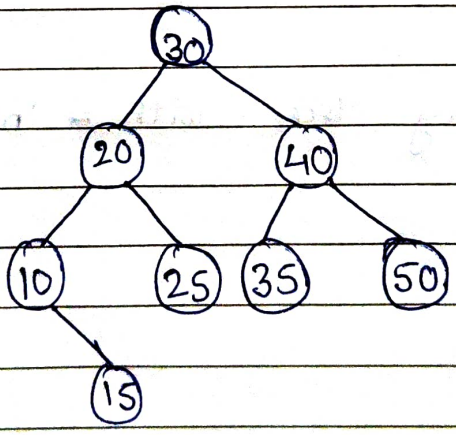
Step 1:



Step 2:



Step 3:



Postorder Traversal: 15, 10, 25, 20, 35, 50, 40, 30

Sundaram
FOR EDUCATIONAL USE

Q.2] A strictly binary tree (or full binary tree) has a non-leaf (internal) nodes. Find the total number of leaf nodes in this tree. Support your answer by proper derivation. (CLO4.1).

Ans

In a strictly binary tree, every internal node has exactly 2 children.

Let, No. of internal nodes = n.
No. of leaf nodes = l.

Total no. of nodes = n + l.
Total no. of edges = (n+l) - 1.

∵ Each internal node has 2 children.
No. of edges = 2n.

$\therefore 2n = (n+l) - 1$

$\therefore 2n = n + l - 1$

$\therefore l = 2n - n - 1$

$\therefore \boxed{l = n + 1}$

∴ A strictly binary tree with 'n' non-leaf nodes has 'n+1' leaf nodes.

Q.3] Write following functions for binary search tree: (i) countNodes() (ii) countLeaf() (iii) countBranch() (iv) findDepth() (v) descending(). (L04.2).

Ans

```
#include <stdio.h>
#include <stdlib.h>

typedef struct node
{
  int data;
 struct node * left , * right;
} node;

typedef struct
{
   node * root;
} BST;

void insert(BST *t , int ele)
{
  node * p , * q;
 p=(node *)malloc (sizeof(node));
  p->data = ele;
  p->left = p->right = NULL;

if(t->root == NULL)
{
   t->root = p;
    return;
}
```

Sundaram
FOR EDUCATIONAL USE

```
q = t->root;
while (q != NULL)
{
  if (ele <= q->data)
  {
    q->left = p;
    return;
  }
  else
    q = q->left;
}
else
{
  if(q->right == NULL)
  {
    q->right = p;
    return;
  }
  else
    q = q->right;
}
}
}

int countNodes (node *t)
{
 if (t==NULL)
   return 0;
return 1 + countNodes (t->left) + countNodes (t->right);
}
```

Sundaram
FOR EDUCATIONAL USE

```
int countLeaf(node *t)
{
  if(t==NULL)
     return 0;
if(t->left==NULL && t->right==NULL)
   return 1;
return countLeaf(t->leaf) + countLeaf(t->right);
}
int countBranch(node *t)
 {
    if(t==NULL)
       return 0;
if(t->left==NULL && t->right==NULL)
       return 0;
return 1 + countBranch(t->left) + countBranch(t->right);
}
int findDepth(node *t)
{
   int ld, rd;
 if(t == NULL)
     return 0;
ld = findDepth(t->left);
rd = findDepth(t->right);

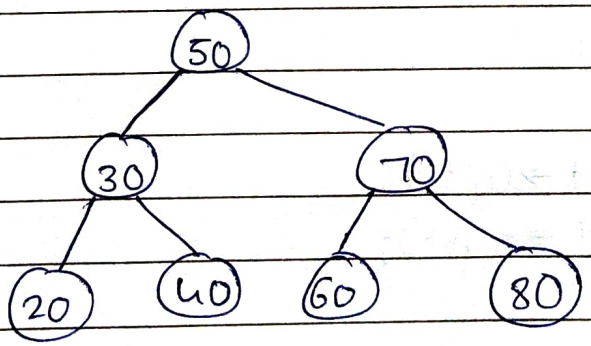
if(ld > rd)
    return ld + 1;
 else
    return rd + 1;
}
```

Sundaram
FOR EDUCATIONAL USE

```
void descending (node *t)
{
  if (t==NULL)
    return;
  descending (t->right);
  printf("%d", t->data);
  descending (t->left);
}
```

Sample Output :-

```
Enter no. of nodes: 7
Enter elements: 50 30 20 40 70 60 80
Total nodes: 7
Leaf nodes: 4
Branch nodes: 3
Depth of tree: 3
Elements in descending order: 80 70 60 50 40 30 20
```

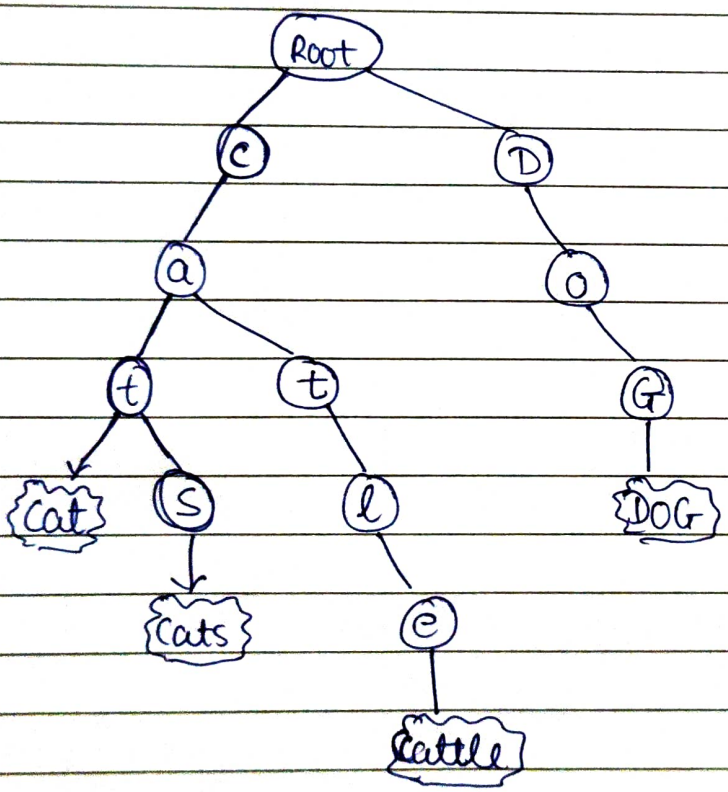


Sundaram
FOR EDUCATIONAL USE

Q4] When typing into a search engine or messaging app, the system suggests completions in real-time. For instance, typing "cat" instantly suggests "cats", "cattle" and "cathedral". Design a data structure that can store a vocabulary dictionary and return all words matching a given prefix. Why is a standard Binary Search Tree (BST) or a Hash Map inefficient for this specific search task? Provide the structural solution. (LO4.2, LO5.1).

Ans

The data structure used for this is Trie also known as Prefix Tree.



FOR EDUCATIONAL USE

Why this is not used?

- BST is designed for ordered search, not prefix search. It may require traversing multiple nodes and does not naturally store common prefixes.
- Hash map provides exact key lookup, but does not group keys with the same prefix. All matching words may be in different hash locations.

∴ Trie, is the best solution as it stores characters along paths and allow efficient prefix search.

Sundaram
FOR EDUCATIONAL USE